Wait-free augmented static trie, recursive form

keys are $1 \ldots N$, with N a power of two
every node reached with bounds i and j covers exactly the keys $i \ldots j$
the fields a value was built with are read back with a dot, as in $x.sum$
an internal node has the form $C[X, Y]$: one node with two recursive children X and Y

```
   type versiontree:                                          ▷ one of two shapes, immutable once built
1      vleaf(nat i, nat sum)                                  ▷ sum is 1 if i is in the set, else 0
2      vnode(nat i, nat j, nat sum, versiontree left, versiontree right)

   type node:                                                 ▷ the skeleton, only its versions change
1      leaf(nat i, versiontree version)                       ▷ version is the one mutable field
2      internal(nat i, nat j, versiontree version, node left, node right)

   versiontree snapshot(node x):                              ▷ atomic read of the x's version
1      return read x.version

   node build(nat i, nat j):                                  ▷ built once, with every sum at 0
1      if i = j
2          return leaf(i, vleaf(i, 0))
3      l ← build(i, (i + j)/2)
4      r ← build((i + j)/2 + 1, j)
5      v_l ← snapshot(l)
6      v_r ← snapshot(r)
7      return internal(i, j, vnode(i, j, 0, v_l, v_r), l, r)

   Initialization:
1      node root ← build(1, N)                                ▷ before any thread runs

   bool refresh(node x, nat i, nat j):                        ▷ one node reads its two children
1      old ← read x.version
2      vl ← snapshot(x.left)
3      vr ← snapshot(x.right)
4      return CAS(x.version, old, vnode(i, j, vl.sum + vr.sum, vl, vr))

   bool insert(node x, nat i, nat j, key k):                  ▷ add k, true iff k was absent
1      if i = j
2          old ← read x.version
3          if old.sum = 1
4              return false
5          return CAS(x.version, old, vleaf(i, 1))
6      if k ≤ (i + j)/2
7          result ← insert(x.left, i, (i + j)/2, k)
8      else
9          result ← insert(x.right, (i + j)/2 + 1, j, k)
10     if not refresh(x, i, j)
11         refresh(x, i, j)                                   ▷ two attempts always suffice
12     return result
```

```
bool delete(node x, nat i, nat j, key k):                ▷ as insert, but clears the leaf
1   if i = j
2       old ← read x.version
3       if old.sum = 0
4           return false
5       return CAS(x.version, old, vleaf(i, 0))
6   if k ≤ (i + j)/2
7       result ← delete(x.left, i, (i + j)/2, k)
8   else
9       result ← delete(x.right, (i + j)/2 + 1, j, k)
10  if not refresh(x, i, j)
11      refresh(x, i, j)
12  return result

bool find(key k):                                        ▷ is k in the set
1   return find_at(snapshot(root), 1, N, k)

bool find_at(versiontree v, nat i, nat j, key k):        ▷ plain sequential code, v is frozen
1   if i = j
2       return v.sum = 1
3   if k ≤ (i + j)/2
4       return find_at(v.left, i, (i + j)/2, k)
5   return find_at(v.right, (i + j)/2 + 1, j, k)

nat size():                                              ▷ answered by a single read
1   return snapshot(root).sum

int select(nat r):                                       ▷ the rth smallest key, −1 if none
1   v ← snapshot(root)
2   if v.sum < r
3       return −1
4   return select_at(v, 1, N, r)

key select_at(versiontree v, nat i, nat j, nat r):       ▷ assumes r ≤ v.sum, so it finds one
1   if i = j
2       return i
3   if r ≤ v.left.sum
4       return select_at(v.left, i, (i + j)/2, r)
5   return select_at(v.right, (i + j)/2 + 1, j, r − v.left.sum)

nat rank(key k):                                         ▷ how many keys are at most k
1   return rank_at(snapshot(root), 1, N, k)

nat rank_at(versiontree v, nat i, nat j, key k):         ▷ the same count, taken inside v
1   if i = j
2       return v.sum
3   if k ≤ (i + j)/2
4       return rank_at(v.left, i, (i + j)/2, k)
5   return v.left.sum + rank_at(v.right, (i + j)/2 + 1, j, k)
```